

Responsible and Sustainable AI

Metro Interstate Traffic Volume — Bias, Fairness, Governance & Sustainability

Prepared by Meenal | September 2026

This report covers the two responsible-AI requirements for the capstone: a bias and fairness assessment of the data and models built across Tasks 1, 4, 5, and 6 (the supervised models, MLflow tracking, recommendation system, and deployment simulation), and a short discussion of the governance and sustainability considerations those same choices raise.

1. Bias and Fairness Report

1.1 Sampling and coverage limitations

The dataset's coverage is narrower than it might first appear, in ways that limit what any model trained on it can honestly claim:

- **Single corridor, single location.** Every record describes one road (the Metro Interstate 94 corridor) at one fixed point. Nothing in this project generalises to other roads, intersections, or regions — the models, the recommendation windows, and the “risk” predictions are all specific to this corridor's own historical behaviour.
- **An arbitrary choice baked into deduplication.** This dataset has a known quirk of multiple rows sharing one timestamp, from overlapping weather-station reports. The pipeline keeps the first occurrence and discards the rest, with no criterion for which station's reading is more accurate an undocumented, arbitrary choice about whose weather reading counts, silently baked into every downstream weather feature.
- **Known, but possibly not exhaustive, sensor faults.** The raw data contains two well-documented error patterns — 0 K temperature readings (which convert to an impossible -273.15°C) and rainfall values in the thousands of millimetres per hour — both detected and corrected with monthly-median imputation. Their presence confirms the underlying weather instrumentation is not fully reliable; other, less obviously “impossible” errors may remain undetected and uncorrected in the data used to train and evaluate every model.
- **Sharply uneven weather-category coverage.** Across roughly six years of hourly data, “Squall” appears only 4 times and “Smoke” only 20 times, versus 15,000+ for “Clouds” and 13,000+ for “Clear.” Any prediction made under these rare conditions is close to extrapolation — there is almost no historical evidence behind it, however confident the model's output looks.
- **Narrow holiday coverage.** The `holiday` field covers only a fixed list of US federal-style holidays (e.g. Columbus Day, Veterans Day, Thanksgiving, State Fair). It has no visibility into other events that plausibly move traffic just as much school breaks, local festivals, sporting events, road closures or construction none of which are flagged anywhere in the data.
- **A fixed historical window.** The data spans a specific multi-year period ending in 2018. Every model reflects driving patterns, vehicle mix, and road conditions from that window; later years — and any real change in the corridor's infrastructure or typical traffic since — are not represented at all. This is a coverage gap in time, not only in geography, and Section 1.3 shows a concrete instance of it.
- **Extreme events are flattened, not removed.** Statistical outliers in traffic volume are capped to the IQR bounds rather than deleted, deliberately preserving genuine extreme events as real rather than discarding them as noise. But capping also numerically flattens the most unusual real hours — a major incident causing near-zero or exceptionally high flow — toward the bulk of the distribution, understating exactly how extreme they were.

1.2 Proxy labels and their risks

No accident dataset was available for this capstone. Every classifier, recommendation, and “risk” prediction built on **accident_risk** (Tasks 1, 4, 5, and 6) instead uses a proxy: an hour is labelled high-risk when **both** traffic_volume is in the top third of observed volumes (congested) and the weather that hour is adverse (severe weather, precipitation, or heavy cloud cover). This is a deliberate, documented substitute for real accident data — and it carries real risks that are worth stating plainly rather than leaving implicit:

- **It measures a definition, not an outcome.** The label encodes an assumption — that congestion plus bad weather predicts real accident risk — that was never checked against an actual accident record, because none exists here. If that assumption is wrong, or only partly right (some traffic-safety research associates heavy congestion with slower, more cautious driving, and so with fewer severe accidents, not more), the label could be systematically miscalibrated, and every model trained on it — including the one served by Task 6's deployment API — would inherit that miscalibration silently, with no way to detect it from this data alone.
- **It's coupled to the regression target it's meant to be independent of.** The label is derived directly from traffic_volume via a quantile cutoff specific to this corridor, while traffic_volume is simultaneously the regression target. The feature set was deliberately built to exclude traffic_volume (and anything derived from it) to avoid the models trivially “cheating” on that leakage — but the conceptual coupling remains anything that shifts what counts as “congested” for this corridor (e.g. a genuine long-run change in typical volume) redefines what counts as “risky,” with no change in actual road safety.
- **The definition, not reality, sets the positive rate.** How often the label fires is a direct function of this project's own threshold choices — a two-thirds traffic quantile, a specific keyword list for “severe” weather, a 75%-cloud-cover cutoff for “overcast.” Different, equally reasonable thresholds would flag a materially different set of hours. The label should be read as “hours meeting this project's operational definition,” not as a validated estimate of which hours are more dangerous.
- **Downstream systems inherit the proxy's authority without inheriting its caveats.** Task 5's plain-language travel recommendations and Task 6's deployed `/predict` and monitoring endpoints all build on this same proxy. A person reading “this window carries elevated risk” has no way to know, from that sentence alone, that “risk” here is a proxy definition rather than measured accident history — a gap addressed further in Section 2.1.

1.3 Uneven error distribution across conditions and time periods

None of the notebooks in this project broke their headline metrics down by weather category, season, or holiday status — every reported MAE, accuracy, precision, and recall is a single aggregate number. That aggregate hides real unevenness this project's own data and its own monitoring already point to:

- **By weather condition.** With only 4 historical Squall hours and 20 Smoke hours, no metric computed on this data can reliably estimate model error under those conditions — the aggregate error is overwhelmingly set by the common categories (Clouds, Clear, Mist, Rain) and says essentially nothing trustworthy about the rare ones. A user relying on this system during a rare weather event has the least evidence behind the prediction they're seeing, at exactly the moment a reliable one would matter most.
- **By time and season — an effect this project actually observed, not just a theoretical risk.** Task 6's own monitoring demonstration (section 6.4, “Demo B — real data, one year later”) found a statistically significant shift in the temperature distribution between a recent reference window and data from one year later (Kolmogorov–Smirnov $p \approx 0.002$), even though the deployed model's prediction error on that same later data did not measurably degrade on that occasion. That combination — real feature drift without (yet) visible error degradation — is exactly the scenario where aggregate error looks fine while something is genuinely changing underneath it.
- **By holiday status.** `is_holiday` hours are a small minority of the roughly six years of data. The model has far less evidence to learn holiday-specific patterns than ordinary-weekday ones, so holiday predictions are plausibly less reliable — a gap this project flags but never actually measured with a holiday-specific metric.
- **What this means in practice.** A single aggregate MAE (≈ 271 for the Random Forest regressor on Task 6's held-out chronological split) should not be read as uniform across conditions. Before this system informs any real decision, error should be computed and monitored per segment — by weather category, by season, by

holiday status — not only as one overall number, precisely because this project's own evidence suggests it is not evenly spread.

2. Governance and Sustainability

2.1 Oversight before this is trusted for real-world decisions

Task 6 already builds several of the right mechanical pieces — versioned experiments, a model registry, and a monitoring/alerting check. None of those are oversight by themselves; they need a human process wrapped around them before any of this should inform an actual decision:

- **Keep production promotion a human decision.** Task 6's registry promotes a version to “Production” explicitly, not automatically — that should stay a deliberate choice informed by more than the holdout MAE/R² alone (per-segment performance, the proxy-label caveats above), not a pipeline that auto-promotes whichever run scores best.
- **Name an owner and a response process for alerts.** A PASS/ALERT status is not oversight on its own. Before this is trusted, a specific person or team needs to own investigating every ALERT, with a defined menu of responses (roll back to the previous registered version, retrain, or accept and document the drift as benign) and a timeliness expectation — otherwise alerts accumulate unread.
- **Make the proxy-label caveat impossible to miss.** Any interface built on this system — Task 5's recommendation sentences, Task 6's `/predict` responses — should prominently and permanently disclose that “accident risk” is a proxy definition, not measured accident data, and that every finding is specific to this one corridor and this historical window. Stating that once in a notebook markdown cell, as this project currently does, is not enough once real people might act on the output.
- **Validate the proxy before any safety-relevant use.** Because `accident_risk` was never checked against real accident records, the single most important step before this system informs anything safety-relevant — signage, road design, enforcement, or emergency response — is sourcing real accident data for this corridor and validating, or correcting, the proxy against it, rather than trusting its current definition indefinitely.
- **Decide a retraining and drift-response cadence in advance.** Given Task 6's own monitoring already found a real one-year seasonal shift, there should be an agreed policy for how often the model is refreshed with new data and what evidence justifies retraining versus leaving the current version in place — decided ahead of time, not improvised after an alert.
- **Preserve the version audit trail as a governance record, not just a debugging aid.** MLflow's registry already gives a reproducible record of every version's parameter, metrics, and artifact. That trail should be treated as a record of who approved which version and why, not only as something to consult when something breaks.

2.2 Environmental and resource trade-offs

Several modelling choices in this project trade accuracy against compute cost — in one case that trade-off was already made explicitly; the rest are worth stating just as plainly:

- **A trade-off already made, not just theoretical.** Task 6's own version comparison found a deeper Random Forest candidate (300 trees, depth 18) improved holdout MAE by well under 1% over the simpler configuration already used in Task 1 (200 trees, depth 12) — and the simpler model was chosen specifically because that marginal gain didn't justify the added training and inference cost. That's a concrete, already-applied instance of weighing accuracy against resource cost, rather than defaulting to “bigger model, better.”
- **Cheap-and-worse versus heavy-and-better.** The Linear Regression baseline trains almost instantly with negligible inference cost, but its MAE (≈ 800) is markedly worse than the Random Forest's (≈ 270) on this project's own held-out data. Which one is the right choice depends on how much that accuracy gain is actually worth for the decision it informs — not on reflexively picking whichever model scores best on paper.
- **A deep-learning option was explored, but not what's actually deployed.** Task 3 investigated an LSTM for sequential traffic prediction and used SHAP explainability on a Random Forest as a lighter-weight proxy for it. Both LSTM training (many epochs over sequential data) and SHAP's exact computation are meaningfully

more compute-intensive than the tree ensembles actually put behind the deployment API in Task 6 — a resource-conscious choice worth stating explicitly rather than leaving it implicit in which notebook happens to serve the API.

- **Retraining is repeated across the pipeline more than a production system would need.** Because Tasks 1, 4, 5, and 6 each independently retrain their own copy of essentially the same Random Forest (an 80/20 split in Task 1, four separate MLflow-logged runs in Task 4, a full-data refit in Task 5, and three candidate versions plus another full-data refit in Task 6), the cumulative training compute across this project is considerably higher than a single production training run would require. That's a reasonable cost during development, but an actual deployment should consolidate to training once, or on a defined schedule, rather than repeating full retraining in every downstream notebook indefinitely.
- **Unmanaged experiment storage grows without bound.** Every MLflow-logged run persists a full serialized copy of its model artifact (tens of megabytes per Random Forest of this size in this project's own testing). Left unmanaged, a tracking store that keeps every historical run and version indefinitely grows without limit — the retention/archival policy named in Section 2.1 should cover storage, not only which version is “current.”
- **Match monitoring frequency to how fast the thing being monitored changes.** Task 6's drift checks are cheap to compute on a batch of recent records, so running them on a periodic schedule (daily or weekly, as demonstrated) rather than on every single prediction is sufficient for a slowly-changing phenomenon like road traffic — real-time, per-prediction monitoring would add continuous compute cost without a matching benefit here.